Name of the Student	
Reg. No	
GitHub Link	

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 3

Comparative Analysis

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0405**
SLOT :D

COURSE OUTCOME COVERED

CO3: Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications. (BTL-6)

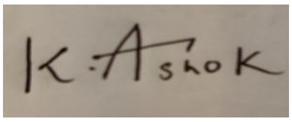
CO Weightage: 25% **Duration: 90 minutes**

Assessment Tool Weightage: 40% **Marks: 25**

Code of Conduct:

I K Ashok/192425322 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Name of the student:K Ashok

Criteria	Selection of Studies/Parameters (2)	Comparison & Differentiation (2.5)	Critical Evaluation (2.5)	Synthesis & Insight Generation (2)	Clarity & Presentation *(1)*	Total (10)
Weightage	*20 %*	*25%*	*25%*	*20 %*	*10 %*	100 %
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

Signature of the Faculty: ***Total Marks :***

QUESTIONS: 05 **Total Marks:25**

Q1. CNN vs Fully Connected Neural Network

A medical imaging organization wants to classify X-ray images using deep learning. Compare a **CNN with a traditional fully connected neural network** in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.

1. Architectural Design

Fully Connected Neural Network:

A traditional fully connected neural network connects every neuron in one layer to every neuron in the next layer. For an image, the pixels are usually flattened into a one-dimensional vector before being given to the network.

CNN:

A Convolutional Neural Network contains convolutional layers, activation functions, pooling/downsampling layers and classification layers. It processes images as two-dimensional or three-dimensional structures and preserves spatial relationships between nearby pixels.

2. Motivation

Fully connected networks can learn patterns, but they require a very large number of parameters when the input is a high-resolution image.

CNNs were developed to efficiently process image data. They automatically learn visual features such as edges, textures, shapes and complex structures.

For X-ray classification, this is important because disease-related patterns may appear in speciic spatial regions of the image.

3. Spatial Feature Extraction

Fully Connected Network:

After flattening the image, the direct spatial relationship between neighboring pixels is not explicitly preserved by the architecture.

CNN:

Convolutional filters scan local regions of the image. Therefore, the network can learn spatial patterns such as edges, textures and shapes while maintaining their relative locations in feature maps.

4. Number of Parameter

Consider an X-ray image of size 224 x 224 pixels with one grayscale channel.

Number of input pixels

224 x 22 = 50,176 pixels

If these pixels are connected to 1,000 neurons in a fully connected layer:

50,176 x 1,000 = 50,176,000 weights

This creates a very large number of parameters.

A CNN cn use a 3 x 3 filter. A single filter has only:

3 x 3 x 1 = 9 weights, plus one bias.

If 32 filters are used:

3 x 3 x 1 x 32 = 288 weights, plus biases.

Therefore, CNNs require far fewer parameters in the feature-extraction stage.

5. Computational Complexity

Fully connected networks require many multiplications because every input is connected to every neuron. This increases memory usage and computation.

CNNs perform local convolution operations and reuse the same filters across the image. This reduces the number of trainable parameters and makes image processing more efficient.

Conclusion:

For X-ray image classification, CNNs are clearly more appropriate than traditional fully connected networks. They efficiently extract spatial features while using parameter sharing and pooling to reduce computational requirements.

Q2. Different CNN Layers and Filters

A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of **convolutional layers, pooling layers, activation layers, and fully connected layers** in terms of feature extraction, spatial information, computational requirements, and classification. Further compare **early-layer filters and deeper-layer filters** based on the type of features they learn, and justify how these layers collectively improve defect detection.

A manufacturing company wants to detect defects in industrial products using a CNN. Different CNN layers perform different but complementary functions.

1. Convolutional Layers

Convolutional layers are mainly responsible for feature extraction.

They apply learnable filters to local regions of an image. These filters can detect:

- Edges.

- Lines.

- Corners.

- Textures.

- Surface patterns.

- Defect shapes.

Early convolutional layers detect simple features, while deeper layers combine these features to identify more complex defect patterns.

2. Pooling Layers

Pooling layers reduce the spatial dimensions of feature maps.

Common types include:

- Max pooling.

- Average pooling.

Max pooling selects the maximum value from a local region.

Pooling helps:

- Reduce computational requirements.

- Reduce feature-map dimensions.

- Reduce the amount of information that must be processed.

- Provide some tolerance to small translations or changes in position.

However, excessive pooling can remove important small defect information. Therefore, pooling must be selected carefully in industrial inspection.

3. Activation Layers

Activation functions introduce non-linearity into the network.

ReLU s commonly used:

ReLU(x) = max(0, x)

Without activation functions, a network made only of linear operations would have limited abilit to learn complex relationships.

Activation layers therefore help the CNN learn complicated defect patterns.

4. Fully Connected Layers

Fully connected layers are usually placed near the end of a traditional CNN classifier.

They combine the high-level features learned by convolutional layers and use them to make a final classification.

For example:

- Normal product.

- Cracked product.

- Scratched product.

- Damaged product.

5. Comparison of CNN Layers

Convolutional Layers:

Main purpose: Feature extraction.

Spatial information: Preserved in feature maps.

Computation: Local operations with shared filters.

Role in defect detection: Detect edges, textures and defect patterns

Pooling Layers:

Main purpose: Downsampling.

Spatial information: Reduced but important features retained.

Computation: Reduces spatial size and later computation.

Role in defect detection: Helps focus on important features while reducing processing requirements.

Activation Layers:

Main purpose: Non-linearity.

Spatial information: Usually maintains the feature-map structure.

Computation: Relatively simple element-wise operation.

Role in defect detection: Enables learning of complex patterns.

Fully Connected Layers:

Main purpose: Final classification.

Spatial information: Usually becomes less explicit after flattening or feature aggregation.

Computation: Can have many parameters.

Role in defect detection: Combines learned features to determine the final class.

Q3. Parameter Sharing vs Independent Weights

A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare **parameter sharing in CNNs with independent weights in fully connected networks** in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of convolutional filter weights improves learning efficiency and justify its importance for large-scale image-processing applications.

A smart surveillance system processes thousands of high-resolution images. Efficient parameter usage is therefore extremely important.

1. Independent Weights in Fully Connected Network

In a fully connected network, every input pixel can have a separate connection to every neuron.

For an image with a large number of pixels, this produces a huge number of trainable weights.

For example, an image containing 1,000,000 input values connected to 1,000 neurons would require approximately

1,000,000 x 1,000 = 1,000,000,000 weights

This requires a large amount of memory and computation.

Q4. AlexNet vs ResNet

A computer vision company is developing an image classification system and is considering **AlexNet and ResNet** as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.

AlexNet generally follows:

Input

↓

Convolution

↓

ReLU

↓

Pooling

↓

Convolution

↓

ReLU

↓

Pooling

↓

Fully Connected Layers

↓

Output

ResNet follows repeated residual blocks:

Input

↓

Convolutional layers

↓

Residual blocks with skip connections

↓

Deeper feature extraction

↓

Global feature aggregation

↓

Output

Q5. Regularized CNN vs Non-Regularized CNN

An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a **CNN without regularization and a CNN using dropout, data augmentation, and batch normalization** in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.

An agricultural organization develops a CNN to classify plant diseases from leaf images. The model has very high training accuracy but poor performance on unseen images. This is a typical example of overfitting.

A suitable pipeline can be:

Leaf Images

↓

Resize and Normalize

↓

Data Augmentation

↓

Convolutional Layer

↓

Batch Normalization

↓

ReLU

↓

Pooling/Downsampling

↓

More Convolutional Blocks

↓

Global Average Pooling or Flatten

↓

Dropout

↓

Output Layer

↓

Plant Disease Class

Conclusion:

A CNN without regularization may achieve very high training accuracy but poor unseen-image performance because of overfitting. A regularized CNN using data augmentation, dropout and batch normalization can generalize better.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand